Assignment 4: Relational Database vs. NoSQL

Comparison, use cases and examples

Project Topic: Hotel Management System

1. Introduction

Relational databases and NoSQL databases both store data, but they organize and manage it differently. The right choice depends on the structure of the data, consistency needs, expected workload, scale and the skills of the organization. A Hotel Management System is a good fit for a relational database because its reservations, rooms, guests and payments have clear relationships.

2. Comparison

Factor	Relational database (RDBMS)	NoSQL database
Data model	Tables with rows, columns and defined relationships	Document, key-value, wide-column or graph models
Scalability	Often scales vertically; horizontal scaling is also possible with suitable systems	Many systems are designed for horizontal scaling across multiple servers
Consistency	Strong transactional consistency is commonly available, with ACID transactions	Varies by product; some support strong consistency and others offer tunable or eventual consistency
Performance	Efficient for structured data, joins and complex transactional queries	Can be very fast for access patterns matching its data model; performance depends on design
Schema	Usually a defined schema; changes are managed deliberately	Often flexible schema, useful when records differ or evolve frequently
Relationships	Foreign keys and joins naturally support related data	Relationships may be embedded, duplicated or handled by application logic
Typical use cases	Banking, booking systems, payroll, inventory and accounting	Large event streams, flexible product catalogs, caching, content and distributed applications
Examples	MySQL, PostgreSQL, SQL Server, Oracle Database	MongoDB, Cassandra, Redis, Neo4j

3. Why Organizations Choose Different Technologies

Organizations choose based on their actual requirements rather than assuming one technology is always best. They may prefer an RDBMS when they need structured records, reliable transactions, constraints and multi-table reporting. They may choose NoSQL when they need flexible records, very high distributed throughput, or a data model such as key-value or graph. Some organizations use both systems for different parts of one application.

4. Real-World Examples

Example 1: Hotel reservation and payment system

A hotel needs to connect guests, rooms, reservations and payments. A relational database such as MySQL is suitable because foreign keys, transactions and SQL queries help protect booking and payment consistency.

Example 2: Product catalog with flexible attributes

An online store may sell clothing, electronics and furniture with different product attributes. A document database such as MongoDB can store varied product documents without requiring every product to have exactly the same

fields.

Example 3: Caching and quick lookups

A website may use Redis, a key-value store, for temporary session data or frequently accessed values. This is a different need from storing the authoritative reservation and payment records.

Example 4: Large distributed event data

A service that collects events from many servers may consider a wide-column database such as Cassandra when high write volume and distributed availability are important. The choice still depends on its consistency and query requirements.

5. Recommended Choice for This Project

For the Hotel Management System in this project, a relational database such as MySQL is recommended. The schema has clearly related tables, primary and foreign keys, payment records and constraints. SQL also supports the required joins, grouping and aggregate reports. A NoSQL database could be added later for a separate use case, but it is not necessary for the core assignment.

6. Conclusion

RDBMS and NoSQL technologies solve overlapping but different problems. Relational databases are strong for structured data, relationships and transactions, while NoSQL systems offer data-model choices and scaling approaches suited to particular workloads. Organizations should select technology based on consistency, performance, scale and application needs.